

Ch6 Programming Language

- Primitives 原始型別：程式語言中最基礎、最核心的資料型態

特性	原始型別 (Primitives)	複合/參考型別 (Non-Primitives)
例子	<code>int</code> , <code>bool</code> , <code>char</code>	String, Array, Object, Class
儲存內容	實際的數值 (Value)	記憶體位址 (Reference/Address)
儲存位置	通常在 Stack (堆疊)	實際資料在 Heap (堆積)，變數存指標
記憶體佔用	固定大小 (例如 int 佔 4 bytes)	大小動態變化
預設值	通常有預設值 (如 0 或 false)	通常為 <code>null</code>

程式語言發展歷史

1. Machine Language 機器語言

使用 2 進位指令集構成，電腦能直接執行 `Machine Language(hex Op-code)`

2. Assembly 組合語言

`Assembler` 組譯器

將機器語言的指令使用助記符號表示(比較好記)，與機器語言 1-1 對應

多了 `Identifier` (可自訂義的名稱、變數)

----- ↑ `Machine Dependent`，工程師要從機器的角度思考，不同電腦間程式不相容 ↑ -----

某些高階語言仍要因為不同平臺進行小修改，不過已經提升很大的相容性

----- ↓ `Machine Independent`，程式設計的方式符合人類的邏輯，不同電腦間程式可相容 ↓ -----

3. 高階語言 (Ex : FORTRAN、COBOL、C、Python、JavaScript...)

`Compiler` 編譯器：把高階語言程式碼轉為 `Machine Language ( C )`

一個高階語言指令對應到多個機器語言指令

`Interpreter` 直譯器：把高階語言程式碼直接逐行執行 (Python)

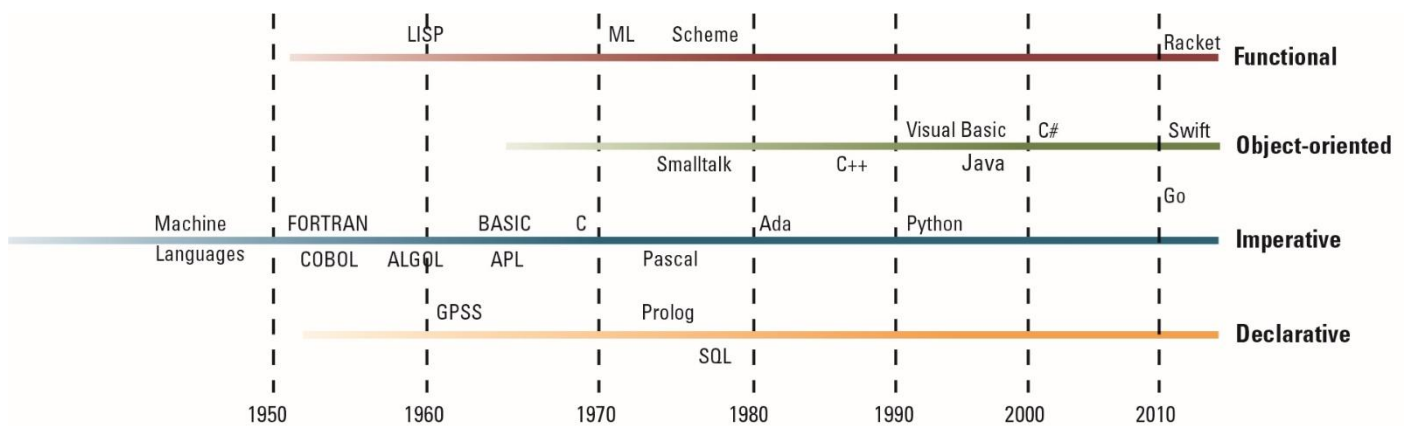
4. 超高階語言 (Ex : SQL、MATLAB...)

相較第 3 代高階語言有明確的應用方向 (資料搜尋、數學計算)

某些高階語言 (如 Python) 已經具備超高階語言的發展特性

5. (AI 自然語言)

Programming Paradigms : 軟體開發時採用的思維



- Functional(FP) : 把程式當數學函數 (不可變動、不依賴外部狀態)
- Object-oriented(OOP)物件導向
- Imperative 命令式：逐步依序執行指令 告訴電腦怎麼做，電腦照做

例：

C

```
x = 0;
for(i=0; i<10; i++) x += i;
```

- Declarative 宣告式：描述執行的目標 告訴電腦你要做什麼，電腦會自己決定要怎麼做來達成

例：SQL

sql

```
SELECT * FROM users WHERE age > 20;
```

你沒有告訴 DB 要 loop 表格，DB 自己決定。

- Event-driven 事件驅動：程式回應事件 (點擊、訊息)

```
btn[0].addEventListener((e)=>{
    alert('pressed');
}, onclick);
```

網頁 JavaScript

Program Unit 一段程式的基本架構

注意：與 Declarative Programming / Imperative Programming 不一樣

- Declarative statements 宣告所需的變數與常數

```
C  
  
int x;    // 宣告一個變數，不執行動作
```

- Imperative statements 依序執行的指令

```
C  
  
x = x + 1;  
i++;  
print("hello");
```

Data Type

- 整數 Integer
- 浮點數 Float (某些語言叫 Real)
- 字元 Character (符號)
- Boolean (True / False)
- Array 陣列

indices 指數 (索引) : array[5]

literal 字面值

在程式碼中直接寫出、代表固定值的符號

```
int age = 30;
```

- 30 就是一個整數字面值。

```
String name = "Alice";
```

- "Alice" 就是一個字串字面值。

```
boolean is_active = true;
```

- true 就是一個布林字面值。 

程式語言語句

- **Assignment** 指定 $z = x + y;$
- **Control** 控制 If...else、While、For、Switch、goto

goto 的方式容易雜亂無章

才發展出 if...else...等 Structured programming(結構式寫法)

- **Comment** 註解



Variables: local v.s. global

- **Scope**：這個變數能被存取的範圍
- **Local variable** 區域變數：宣告在 function 中，只能在 function 中被呼叫的變數
- **Global variable** 全域變數：在程式碼中沒有被限制 scope 的變數

Function Parameters: Formal v.s. Actual

```
int sum(int a, int b) {  
    return a + b;  
}  
sum(x, y);
```

- Formal : 宣告時 function 內使用的 Identifier a, b
- Actual : 實際傳入的變數 x, y

Pass By Value v.s. Pass by Reference

- Value：建立傳入變數資料的副本，把值配給 Formal Parameter

C 語言內建只可 Call by value

優點：安全，原始資料不會被更動

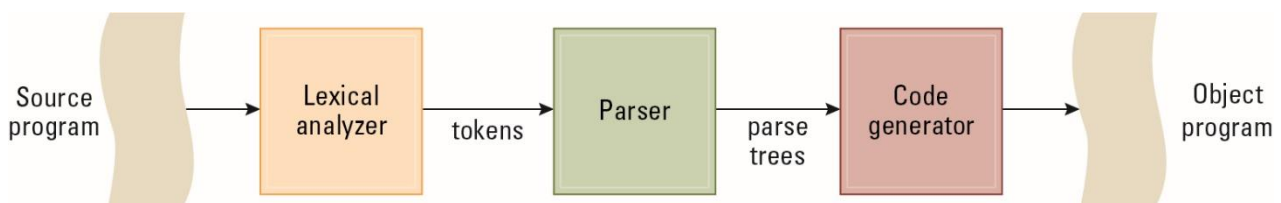
- Reference：傳入變數的地址

C 語言可用傳遞指標模擬 Call By Reference

優點：直接操作原始資料，執行時間較快（省下建立副本的複製時間）

Language Implementation 執行程式

Compile 編譯：把高階語言轉換成機器語言的過程



1. 辨識字串(source code)中的語法單元或 token
2. Parser 將 token 組成 statement(語句)，透過 syntax diagrams(語法圖)建立 parse tree
3. Code Generator 建立 machine language 來執行

程式碼中的各種角色

- Keywords 關鍵字：if else then
- Token：程式碼中的最小單位
- Identifier：程式中用來命名東西的字串

Grammar 文法：用來定義語言結構的規則

- Terminal 終結符號：程式碼中的最小單位(token)
- Nonterminal 非終結符號：語法結構的分類，用於表示語句組成方式。

不會出現在程式碼中，parse 時會被展開

Expression, Statement, Program



Ambiguous Grammar 模稜兩可

對於某些輸入字串，文法可以產生兩個或以上不同的 parse tree，解釋方式有歧義

例：3 + 4 * 5 有先加和先乘兩種解讀方式

Coercion 強型別轉換

語言在「需要某種型別」時，自動把值轉為該型別

例：

```
int i = 3;
double d = i + 0.5;    // i 被升級為 double
```

Strongly Typed 強型別語言

只允許明確定義過合法的 Coercion (例：Python、C)

Python

python

複製程式碼

```
"5" + 1
# TypeError: can only concatenate str (not "int") to str
```

- 相對：Weakly Typed 弱型別語言 (例 JavaScript)

JavaScript

js

```
"5" - 1    // 4
"5" + 1    // "51"
true + 1   // 2
```

型別容易亂轉，造成 bug

Type Promotion 型別提升

語言在運算時，自動把較小精度的型別轉為較大精度，避免資料誤差遺失

`int + float → float`

`float + double → double`

Type Cast 型別轉換 (Explicit Conversion 顯性轉型)

程式設計時主動轉換型態

```
int i = 10;
double d;

// 顯式轉型
d = (double)i;           // int → double
i = (int)3.14;           // double → int, 小數被截掉
```

OOP 物件導向 Object-Oriented Programming

Why OOP?

當程式開發功能越來越大，多人協作開發時容易因為太複雜弄得很混亂，基本的 function style 程式容易被隨意更改，程式碼又不易理解。

因此創造一種概念上更接近現實生活（物件概念），把程式分成好維護的小單位，且更容易擴充的方法。

物件導向的基本概念

- **Class 類別** 模板/類型 大客車
- **Object 物件** class 的 instance(實體) 381-WB 這臺大客車
- **Attribute 屬性** class 或 object 裡面的資料 381-WB 這臺大客車的「出廠年份」是「2012」
 - ✧ **Class Attribute**：只要是這個 Class 值都一樣的屬性 大客車「都是載客用的車」
 - ✧ **Instance Attribute**：每個 instance 都有不同的值 381-WB 這臺大客車「可載客 39 人」
- **Method 方法** class 或 object 裡面的函式 381-WB 這臺大客車的車門開關
- **Constructor** 把 class 實體化且初始化 Attribute 的 method 向主管機關登記一臺新出廠的大客車

物件導向的四大支柱

● Encapsulation 封裝

將資料與函式的實作內容包裝、隱藏起來的技巧

✧ **資料隱藏**的概念：Access Control 存取控制

概念層面 為了不讓物件外依賴或干涉物件內的細節，把細節隱藏起來

水電工在修廁所，屋主可以監工，但不應該插手水電工正在做的東西

語言層面 透過 public、private、protected (Java) 等關鍵字控制物件內的資料安全

✧ Interface 介面

外部存取 透過 Getter、Setter (物件內的 Method) 控制資料的操作

你按電梯按鈕，不需要知道裡面是什麼電路。

物件外只要知道呼叫什麼方法可以做什麼事，不需要知道方法如何實作的。

```
java

class Person {
    private int age;          // 封裝的資料

    public int getAge() { // 封裝的介面
        return age;
    }

    public void setAge(int age) {
        if (age >= 0) this.age = age; // 控制資料安全
    }
}
```

● Inheritance 繼承

原先定義好的 class，可以透過繼承來擴充，新定義的 class 可以完全沿襲被繼承 class 的所有

Attribute 和 Method

python

```
class Dog(Animal):  
    def speak(self):  
        print("Woof")
```

java

```
class Dog extends Animal {  
    @Override  
    void speak() {  
        System.out.println("Woof");  
    }  
}
```

生物都有生命現象，動物、植物則繼承生物，發展出不同的特質

● Abstraction 抽象

✧ Abstract method 抽象方法

可在宣告 class 時只宣告 Method 而不具體實作

✧ Abstract class 抽象類別

Class 中有 Abstract method 即為 Abstract class。因為其中有尚未完成的 Method，只能拿來

被繼承，不能初始化

交通工具都有煞車

但實際上在路上跑的是屬於交通工具的汽車、公車、摩托車、腳踏車

找不到真的叫做「交通工具」的東西在路上跑

● Polymorphism 多型

不同 Instance 的 Method 在同樣命名的情況下，呈現不同實作細節的機制，如此可以讓使用者

在同一個操作介面下，實踐不同的運算結果

都是交通工具的煞車，但不同車種的煞車原理不同，但都達到減速的效果

Parallel / Concurrent Processing

- **Parallel**：實際上要多 CPU 同時共同運作

有三個人同時分別在做 Job1、Job2、Job3 的事情

- **Concurrent**：用 Time-sharing 在單一 CPU 模擬多工，但是同一時間只會做一件事

舉例來說一個人可能會去做 Job1、Job2、Job3 這三件事，但沒辦法影分身之術

所以只能早上做 Job1、下午做 Job2、晚上做 Job3 每天不斷的切換

一個月看起來，這個人確實做了三份工作

資料存取控制

- Mutual Exclusion 互斥

鎖的概念，確保同一個時間點只有一個 Process 能夠存取特定資料或執行特定程式碼區段

Process 要自己搶鎖，使用完畢也要由 process 釋放（工程師要設計）

可能因為有 process 沒釋放造成 deadlock

比喻：公共廁所（鑰匙式） 你（Process）要上廁所，必須自己去櫃檯拿鑰匙（搶權），開門進去。上完廁所出來，你必須記得把門鎖好並把鑰匙放回櫃檯（釋放）。如果你把鑰匙帶回家了，後面的人就都別想上了。

- Monitor

管理崗哨的概念，較高階、封裝性較好的存取控制結構

由程式語言或系統自動控制互斥，較不易出錯

比喻：銀行金庫（有警衛） 你（Process）要去金庫拿錢。你不用自己拿鑰匙開門。你只要走到門口跟警衛（Monitor 機制）說「我要拿錢」。警衛會看裡面有沒有人，如果有人，警衛會叫你在旁邊等；如果沒人，警衛會開門讓你進去。你出來時，警衛會自動換下一位。

Logical Deduction 邏輯推理

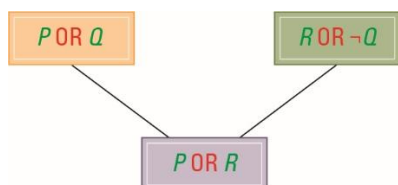
Clause Form

- Literal 一個原子命題或其否定： $P, \neg P$
- Clause 子句：一串由 literal 組成，用 OR ($\vee$) 連接而成的式子： $P \vee Q \vee \neg R$
單一 literal 也算 clause： $P, \neg P$
- Clause Form 子句形式：一串由 Clause 組成，用 AND ($\wedge$) 連接而成的式子：
 $(P \vee \neg Q) \wedge (Q \vee R) \wedge (\neg R)$

Inference Rules

一個保證「前提成立，結論成立」的推理規則

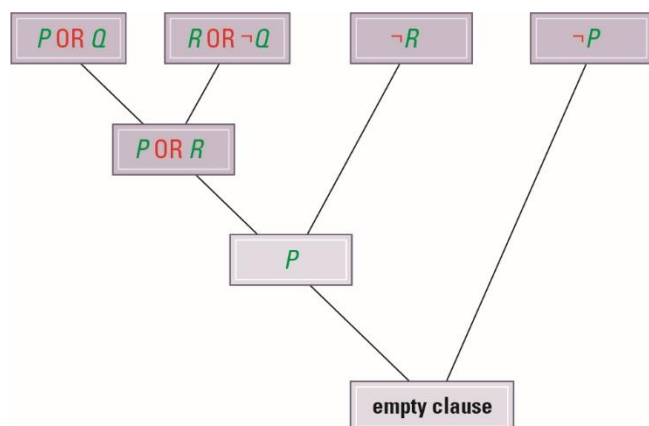
- Resolution 歸結是一種 Reference Rule



Inconsistent 不可滿足

Clause Form 中，沒有任何情況能讓他們同時為真

在 Resolution 能推導出 Empty Clause Form (也就是等價於 False) 就代表 Inconsistent



Prolog = PROgramming in LOGic

- Declarative Programming
- 以邏輯導向的語言，程式就是邏輯知識庫
- 寫什麼是真的，不用寫怎麼推導
- 由三種組成

■ Facts

邏輯的 Literal

human(socrates)

parent(alice, bob)

不需要證明，直接是真的

■ Rules

mortal(X) :- human(X)

human(X) $\rightarrow$ mortal(X)

■ Queries

?- mortal(socrates)

這個敘述能不能從現有知識推導出來？